

What Replit is

Replit is a browser-based coding workspace that lets you **write code** from natural language prompts. Think of it as a ChatGPT trained to create apps and UIs. This is the fastest way to get from an idea to an app.

Similar products: Lovable

Link: <https://replit.com/>

Free trial: limited by the number of apps and AI agent usage

Paid version: if, during the hackathon, you need a paid version, address mentors. They will assist with the purchase.

Common use cases

- **Build relatively simple apps**
 - Building small apps can be with integrations, but the code structure shouldn't be too complex. The more complex it gets, the harder it is to change something without breaking already existing parts.
- **Proofs of Concept (POCs)**
 - Any apps you need for demos, elevator pitches, or verification of hypotheses can be built using Replit.
- **UI and Prototyping (end-to-end)**
 - UI/UX experiments, gathering user feedback, presenting options of UI - all of it can be done via building just a fully clickable UI.

Why people use it

- **Zero setup:** No local installs, works on any device.
- **Fast sharing:** Send a link; others can run or edit instantly.
- **Built-in hosting:** Click to deploy; Replit gives you a URL.

- **No tech background required:** Allows building apps and UI prototypes with 0 tech skills.
- **Batteries included:** Package manager, console, filesystem, simple DB options, and basic logs.
- **Agentic:** Prompts go through agents (architect, developer, QA, etc.) to provide the best result based on very simple prompts.

Strengths vs. limits

- **Strengths:** Speed, collaboration, shareability, great for learning, demos, hackathons, and lightweight services.
- **Limits:** Not ideal for heavy production workloads, strict compliance needs, complex microservices, or advanced observability/CI at scale. You'll eventually want a conventional cloud setup (AWS/GCP/Azure) for large, long-running systems.

Example prompt for simple app:

“Create an app that will parse .csv and .xlsx files and check if the file is valid or not. Validation should happen by comparing column headers to a predefined list of headers. The file may have more columns than specified in the predefined list; however, it must have all the required columns. If they are not present in the file, the file is invalid. If the file has all the required columns, it is valid.

.xlsx files can have multiple sheets, if headers in at least one sheet are valid, the file is valid, but also specify which sheet has valid data.

List of headers:

Date*

Net*

Transaction ID*

TimeZone*

Example of a valid file attached.”